Free the tail, keep the head

Tail-only preemption in vLLM · Qwen3-14B · one H200 · vLLM 0.11.1

Sagnik Biswas

22 October 2026

The bug: 7 minutes becomes 31

Pool at 45% memory: **29.5 GiB** (12,100 blocks of 16 tokens). Each request: 8,064-token prompt, forced to 32,768 output tokens. Caching off.

run	KV needed	preemptions	time
4 requests	24.9 GiB	0	7.1 min
5 requests	31.2 GiB	807	30.7 min

The fifth request overflows the pool by 1.7 GiB, about 6%. Stock vLLM answers by throwing away a *whole* request and recomputing it, over and over.

What stock vLLM does on preemption

- Free *every* KV block of the victim.
- Set its progress to zero and put it back in the queue.
- On resume, prefill the prompt and everything it has generated so far.

The pool was short by a few hundred blocks. The victim held thousands. Most of what gets freed did not need to be freed.

The patch: a scheduler subclass

`serving/tail_preempt_scheduler.py`, passed in through `scheduler_cls`. No vLLM fork.

- On preemption, keep the victim's head blocks pinned. Free nothing yet.
- When someone needs memory, trim the pinned blocks one at a time from the *tail* of the lowest-priority victim.
- On resume, hand the surviving head back. Recompute only the trimmed tail.
- Wait to resume until the request's remaining blocks fit, so it is not preempted again right away.
- Fall back to stock behavior when something is unsupported (spec decode, KV connectors, multiple KV groups, async scheduling).

Works with prefix caching on or off.

Five requests, same pod

caching	scheduler	preemptions	recomputed	time
off	stock	815	31.5M	30.2 min
off	**tail**	**1**	**8.4k**	**8.0 min**
on	stock	1	8.4k	8.0 min
on	tail	1	8.4k	8.0 min
earlier: 4 requests, no pressure		0	0	7.1 min

Every arm reached all 32,768 tokens. With caching off, tail is **3.8× faster** and recomputes 3,700× fewer tokens. With caching on, stock and tail are identical: same one preemption, same 528 blocks recomputed.

Inside the tail run

event	count
preemptions	1
blocks pinned at preemption	2,419
blocks actually trimmed	528
tokens reused on resume	30,256
tokens recomputed	8,449

The pool needed 528 blocks. Tail freed 528 blocks, not 2,419. Stock freed whole requests 815 times and recomputed 31.5M tokens.

The honest caveat

Prefix caching is on by default in vLLM, and it already rescues this case: stock with caching on took **478 s with 1 preemption**, the same as tail.

- Freed blocks keep their hashes, and the free queue evicts the tail first. So stock plus caching recovers most of the head on its own.
- The 31-minute behavior needs caching off.
- What tail adds: the same recovery *without* relying on prefix caching. That matters where caching is off or does not apply (prompt embeddings on vLLM 0.11.0, multimodal inputs, per-request salts).

With caching on, tail adds nothing: stock already recovers exactly the same head.

LatentMAS on AIME: no gain

60 AIME 2024+2025 problems, up to 20k output tokens, the Judger served through vLLM.

concurrency	scheduler	output tok/s	preemptions	recompute tokens
16	stock	844	0	0
16	tail	874	0	0
32	stock	1033	35	94k
32	tail	999	62	149k
60	stock	996	54	240k
60	tail	1002	51	208k

Differences are inside the 3.5% noise floor. Recompute is a cheap prefill here, at most about 4% of time.

Takeaways

- Under real overflow with caching off, tail preemption turns 815 preemptions and 30.2 min into 1 preemption and 8.0 min.
- With caching on, stock gets exactly the same result. Tail only helps where caching is off or does not apply.
- On LatentMAS AIME serving, recompute is too cheap for either scheduler to matter.
- Next: workloads where recompute is expensive and caching does not help, such as long contexts, many victims at once, or caching off by design.